Week 04

Building and Managing Systems

Management of Information Systems (IS) Assessment

Two hours · Three lessons · Three working code examples
Designed for undergraduate Computer Science (CS) students
Online, asynchronous, self-paced, and offline-ready

Orientation — how to use this deck

Lecture §0

The deck is the spine; the markdown lecture is the full read-aloud script.

The core learning package works offline after downloading the files.

Students read the lecture, inspect the diagram/spreadsheet, run or read the code, then answer the checkpoint.

Professor notes are included only in the instructor deck; the student deck removes them.

External videos are candidate links/searches requiring live verification before distribution.

Learning outcomes (1/2)

Lecture §1

Explain the complete system-building process from business need to maintenance.

Distinguish systems analysis from systems design and explain why confusing them creates rework.

Compare development approaches using management criteria, not preference.

Carry out a basic technical, economic, operational, and schedule feasibility assessment.

Convert a project objective into scope, tasks, dependencies, milestones, owners, risks, and measurable status.

Learning outcomes (2/2)

Lecture §1

Explain critical-path logic and use earned-value measures to diagnose schedule and cost performance.

Explain why change control is necessary even when a requested change is reasonable.

Compare centralized, duplicated, decentralized, and networked global-system configurations.

Decide which global-system elements should be standardized and which require localization.

Assess a fictional regional rollout using privacy, language, payments, performance, support, and integration evidence.

Vocabulary and abbreviations — selected working set

Lecture §2

The lecture expands each abbreviation in round brackets when used.

The full downloadable table is reference_table_00_abbreviations.csv.

Focus on the terms you will use in the examples: IS, API, KPI, MVP, SPI, CPI, and SLA.

API	Application Programming Interface	Software-to-software request/data interface
CPI	Cost Performance Index	Earned value / actual cost; below 1.00 is unfavorable
ERP	Enterprise Resource Planning	Integrated software for shared business processes
GDPR	General Data Protection Regulation	EU personal-data regulation
IS	Information Systems	People + process + data + technology
KPI	Key Performance Indicator	Selected measure tied to an objective
MVP	Minimum Viable Product	Smallest usable release that tests key assumptions
SaaS	Software as a Service	Finished software rented over a network

Do not memorise the full table before learning the examples. Use it as a reference while reading and running the files.

LESSON 1

Building Information Systems · What should be built, why, and how do we know it is ready?

Read **Apply** **Assess**

Building Information Systems — what the work includes

Lesson 1 · §3.1

A new IS (Information Systems) project begins with a business need, not with code.

Main activities: identify the problem, analyse requirements, assess feasibility, design the solution, build or configure it, test it, convert to it, operate it, and maintain it.

Programming is only one possible implementation activity.

A purchased package, SaaS (Software as a Service), or low-code workflow can still be a system-building project.

Assessment question: does the system fit the work, the users, the data, the risks, and the operating constraints?

Organizational change — new systems change work

Lesson 1 · §3.2

A system changes how people enter data, approve work, monitor performance, and correct errors.

A technically correct system can fail if the changed workflow is not accepted or supported.

Common change levels: automation, rationalization of procedures, business-process redesign, and paradigm shift.

Newbie assessment rule: ask who must change their daily work, what they stop doing, what they start doing, and who approves the change.

If no one can answer those four questions, the system scope is not ready.

Systems analysis — deciding what is actually required

Lesson 1 · §3.3

Systems analysis asks: what problem exists, what information is needed, who uses it, what process should change, and what rules must the system enforce?

Outputs include requirements, use cases, data needs, process descriptions, constraints, and acceptance evidence.

Separate functional requirements from non-functional requirements.

Functional: what the system does. Non-functional: how well, how safely, how fast, how available, and under what constraints.

Analysis is about the problem and requirements; design is about how to satisfy them.

Feasibility assessment — should the proposal proceed?

Lesson 1 · §3.4

- Technical feasibility: can current technology and staff build, operate, and support it?
- Economic feasibility: do expected benefits justify total cost over the system life?
- Operational feasibility: will users and processes adopt it successfully?
- Schedule feasibility: can it be delivered when the organization needs it?
- Campus Equipment Booking System result: overall weighted feasibility = 4.10 / 5.00.
- Decision: proceed with controlled release-one scope; defer lower-value work.

Technical	4	0.25	1.00	Existing identity service + supported web stack
Economic	4	0.25	1.00	Benefits justify build/support costs
Operational	5	0.30	1.50	Users already use browser-based services
Schedule	3	0.20	0.60	Limit release one; defer dashboard
Overall			4.10	Proceed with controlled release-one scope

Systems design — deciding how requirements will be satisfied

Lesson 1 · §3.5

Design turns requirements into a proposed solution structure.

Design decisions include interface design, process design, data design, security controls, integration, reporting, roles, conversion, training, and support.

A good design is traceable: each important feature points back to a requirement and an acceptance test.

Design also shows trade-offs: speed versus control, customization versus maintainability, local convenience versus standardization.

No design should be approved if nobody can explain how it will be tested.

Choosing a system-building approach

Lesson 1 · §3.6

There is no single best development approach; fit matters.

Traditional SDLC (Systems Development Life Cycle): stronger when requirements are stable and approvals/documentation matter.

Prototyping: useful when users need to react to screens and workflow before requirements are clear.

Agile iterative: useful when priorities change and decision makers can give frequent feedback.

Low-code/end-user development: fast for small workflows but requires governance and support ownership.

Application package/SaaS: useful when standard processes can fit a configurable product.

Outsourcing: useful when specialist capability is missing, if contracts and knowledge transfer are managed.

Traditional SDLC	3.35	Stable requirements; approvals/documentation important
Prototyping	3.85	Users must react to screens/workflow before requirements are clear
Agile iterative	4.05	Changing priorities + frequent user feedback
Low-code / end-user	3.60	Fast small workflows; governance needs attention
Application package / SaaS	3.95	Standard process can fit a configurable product
Outsourcing	3.25	Specialist capability missing; contract/knowledge transfer managed

Testing, conversion, production, and maintenance

Lesson 1 · §3.7

Testing checks that the system works as intended before users depend on it.

Types: unit testing, integration testing, system testing, user acceptance testing, security/control testing, and performance testing.

Conversion choices: direct cutover, parallel operation, phased conversion, and pilot conversion.

Production begins when the organization depends on the system for real work.

Maintenance includes corrections, adaptations, improvements, and support as business conditions change.

Assessment question: what evidence proves the system is ready, and who accepts that evidence?

Real-life example — Campus Equipment Booking Upgrade

Lesson 1 · §3.8

Task: select a release-one scope for a campus equipment booking system.

Relation to the lecture: performs systems analysis, prioritization, feasibility, design traceability, and release planning on one small scenario.

The manual process causes double-booking, lost emails, unclear availability, and inconsistent reminders.

Release-one capacity is limited to 21 effort points.

Mandatory requirements must be included before the release can be approved.

Lower-priority work is deferred rather than silently forgotten.

Release-one requirements and trade-off

Lesson 1 · Example data

Selected release-one scope uses 21 / 21 effort points.

Included: student sign-in, search available equipment, reserve equipment, email reminder.

Deferred: usage dashboard and role-based administration.

Management finding: release one is acceptable because all mandatory requirements are included.

Risk note: deferred role-based administration must be managed before wider administrative use.

ID	Requirement	Priority	Effort	Value	Release 1
R1	Student sign-in	Must	3	10	Yes
R2	Search available equipment	Must	5	9	Yes
R3	Reserve equipment	Must	8	10	Yes
R4	Email reminder	Should	5	7	Yes
R5	Usage dashboard	Could	8	5	No
R6	Role-based administration	Should	5	8	No

Schematic — system-building flow

Lesson 1 · Figure

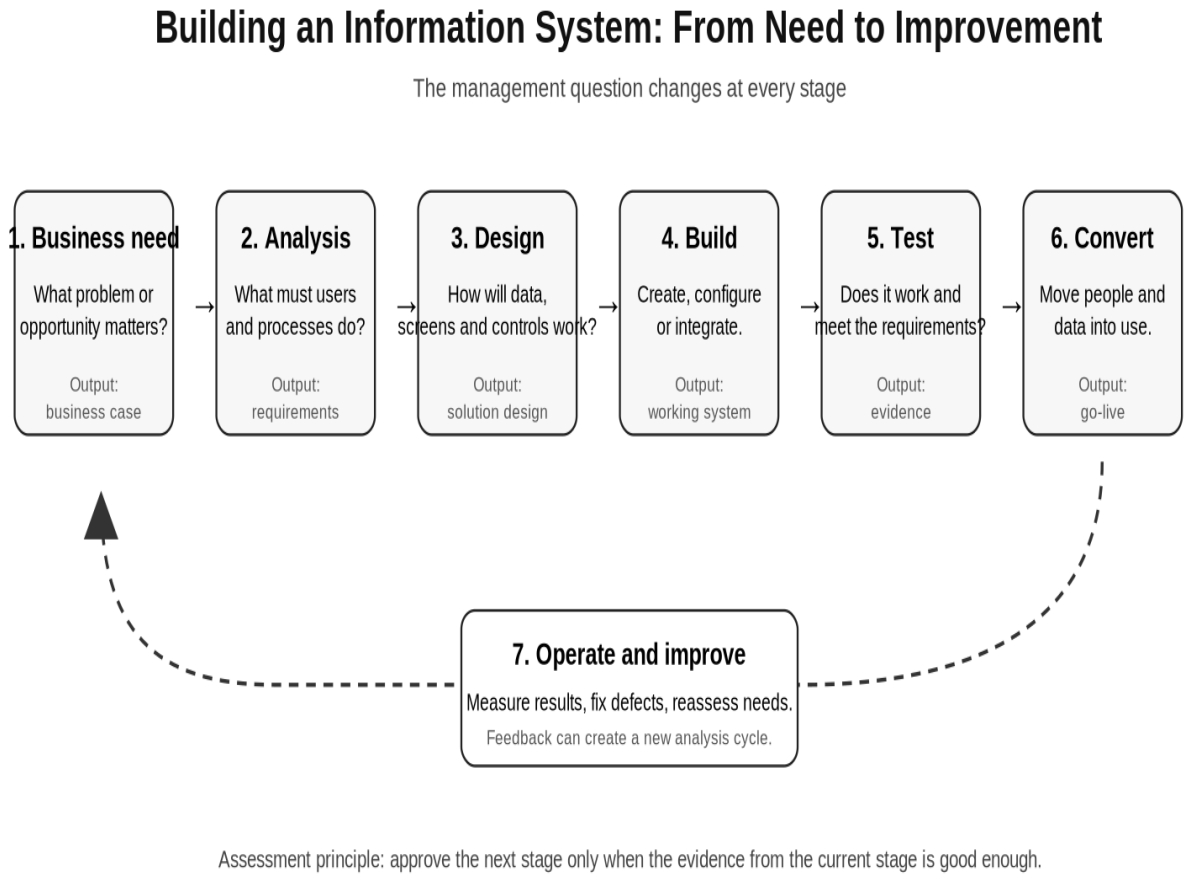
The flow begins with a business need, not with programming.

Requirements and feasibility control whether the work should proceed.

Design translates requirements into a solution structure.

Testing and conversion create evidence before production use.

Maintenance feeds changes back into future requirements.



Code Example 1 — Python release planner

Lesson 1 · Code

```
                                                # Sort requirements by priority first and business value second.
ordered = sorted(requirements, key=lambda item: (priority_rank[item["priority"]], -item["value"]))
# Create an empty list for release-one scope.
selected = []
# Start used capacity at zero.
used_capacity = 0
# Examine requirements in business-priority order.
for item in ordered:
    # Add the requirement only if it fits the remaining release capacity.
    if used_capacity + item["effort"] <= RELEASE_ONE_CAPACITY:
        selected.append(item)
        used_capacity += item["effort"]
```

Task: choose the release-one scope from prioritized requirements and limited capacity.

Relation: analysis and release planning become a checkable calculation.

Full file: code/L1_release_planner.py; every executable line is commented.

Output: selected/deferred work, capacity used, and mandatory-scope check.

Lesson 1 — checkpoint and candidate video

Lesson 1 · §3.9

Checkpoint questions:

1. Why is programming only one part of building an information system?
2. What is the difference between systems analysis and systems design?
3. Which feasibility area is weakest in the Campus Equipment Booking example?
4. Why was the usage dashboard deferred?
5. Why should every design element trace back to a requirement and test?

Candidate short-video search: building information systems / SDLC short lecture on YouTube.

Important: verify availability, duration, publisher, and suitability before distribution.

LESSON 2

Managing Projects · How scope, schedule, cost, risk, quality, and responsibility are controlled

Read **Apply** **Assess**

Why system work is managed as a project

Lesson 2 · §4.1

A project is temporary work with a defined objective, start, end, scope, resources, and accountability.

System work needs project control because it involves uncertainty, coordination, technical dependencies, changing expectations, and limited resources.

Operations repeat; projects create or change something.

Good project management makes trade-offs visible before they become failures.

Assessment question: what is the scope, who owns each task, what evidence shows progress, and who can approve changes?

Scope — what the project is and is not delivering

Lesson 2 · §4.2

Scope defines the agreed work and deliverables.

In scope: functions, data, users, interfaces, reports, controls, training, documentation, and support setup approved for the project.

Out of scope: useful work not included in the current authorization.

A scope statement protects the project from uncontrolled growth and protects students from unclear expectations.

A Work Breakdown Structure (WBS) decomposes scope into manageable work packages.

No clear scope means no fair schedule, no fair budget, and no fair evaluation.

Schedule logic — tasks, dependencies, milestones, critical path

Lesson 2 · §4.3

A task is work that consumes time and has an owner.

A dependency says one task cannot start until another is complete.

A milestone is a zero-duration checkpoint or decision point.

The critical path is the longest dependency chain through the project.

A delay on the critical path delays the whole project unless scope, resources, or logic changes.

Shortening a non-critical task may produce no change in the delivery date.

ID	Task	Pred.	Days	Cost	Status
A	Confirm requirements	-	2	1,200	Complete
B	Design screens	A	3	1,800	In progress
C	Build database	A	4	2,400	In progress
D	Integrate portal	B,C	3	3,000	Not started
E	User acceptance testing	D	2	1,600	Not started

Schematic — project network and critical path

Lesson 2 · Figure

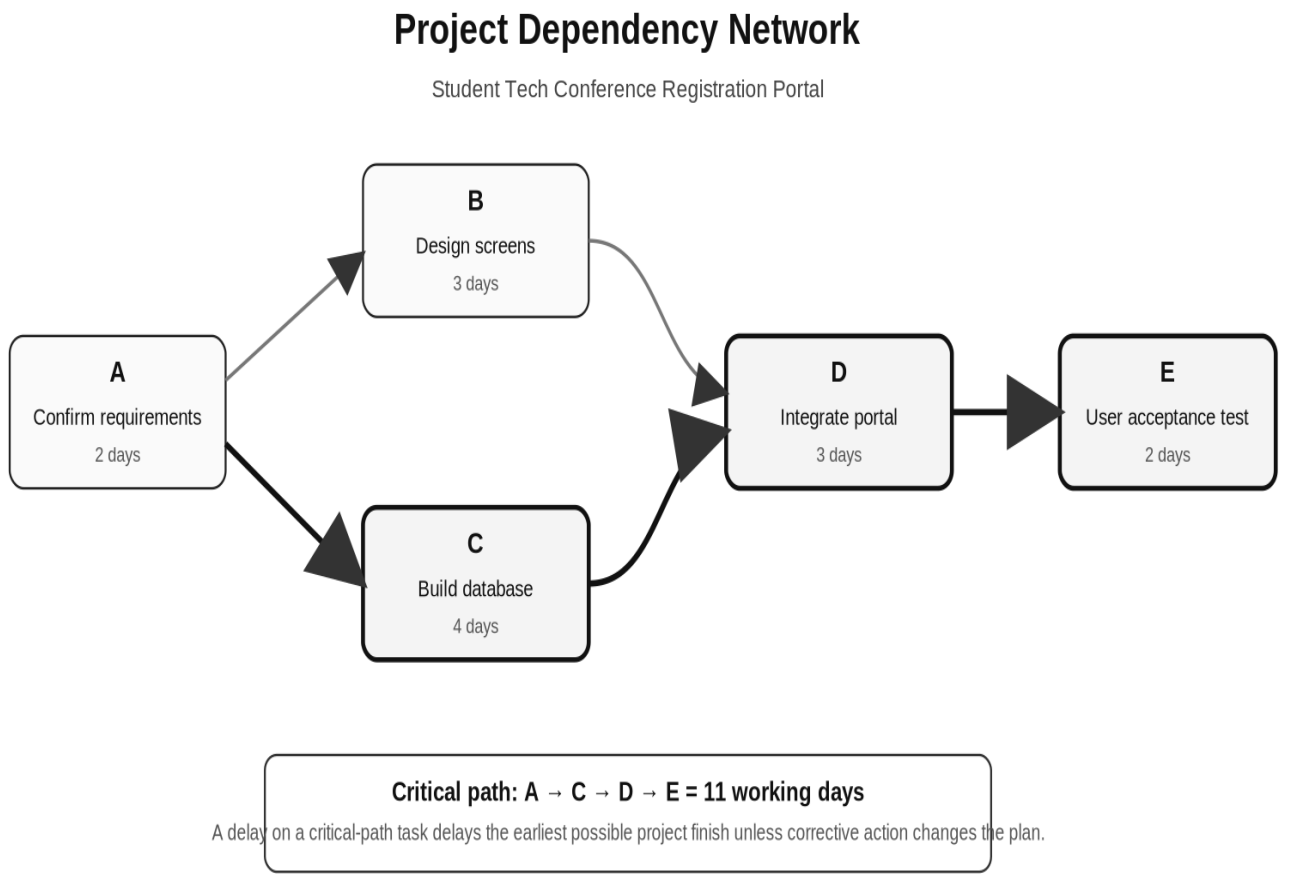
The network shows task order, not just task names.

A → C → D → E is longer than A → B → D → E.

The critical-path duration is 11 days.

Delays in Build database, Integrate portal, or User acceptance testing directly threaten the finish date.

The network makes schedule risk visible before it becomes a missed deadline.



Cost and progress — earned-value diagnosis

Lesson 2 · §4.4

Planned Value (PV): budgeted value of work scheduled by the status date.

Earned Value (EV): budgeted value of work actually completed.

Actual Cost (AC): money actually spent.

Schedule Variance (SV) = EV - PV = CAD -2,400.

Cost Variance (CV) = EV - AC = CAD -1,000.

Schedule Performance Index (SPI) = EV / PV = 0.76.

Cost Performance Index (CPI) = EV / AC = 0.88.

Finding: behind schedule and over budget for completed work.

Measure	Formula	Result	Meaning
PV	planned value	CAD 10,000	Budgeted work scheduled by status date
EV	earned value	CAD 7,600	Budgeted value of completed work
AC	actual cost	CAD 8,600	Money actually spent
SV	EV - PV	CAD -2,400	Behind schedule in value terms
CV	EV - AC	CAD -1,000	Over budget for completed work
SPI	EV / PV	0.76	Less than 1.00 = unfavorable schedule
CPI	EV / AC	0.88	Less than 1.00 = unfavorable cost

Project risk — uncertainty needs an owner and response

Lesson 2 · §4.5

A project risk is an uncertain event or condition that may affect scope, schedule, cost, quality, or acceptance.

Risk exposure = probability × impact.

A risk register should state the risk, owner, response, trigger, and status.

The highest exposure in the scenario is scope expansion after design approval: 4 × 4 = 16.

A risk without an owner is not managed; it is merely noticed.

Risk response choices include avoid, reduce, transfer, accept, or prepare contingency.

Risk	Prob.	Impact	Exposure	Owner	Response
Requirements approval late	3	4	12	Project manager	Decision meeting + deadline
Database takes longer	3	5	15	Technical lead	Early technical spike + daily checks
Staff unavailable for UAT	2	5	10	Business sponsor	Book test sessions early
Scope expands after design	4	4	16	Sponsor	Impact analysis + formal approval

Roles, responsibility, and decision authority

Lesson 2 · §4.6

Project sponsor: owns the business reason and approves major decisions.

Project manager: coordinates scope, schedule, cost, risk, communication, and control.

Business owner or product owner: confirms priority and acceptance from the user side.

Technical lead: owns technical design, estimates, integration, and quality evidence.

End users: test whether the system fits real work.

A RACI (Responsible, Accountable, Consulted, Informed) view prevents confusion about who decides and who only advises.

Change control and quality

Lesson 2 · §4.7–4.8

Change control exists because reasonable individual changes can produce unreasonable total impact.

Every proposed change should state: request, reason, affected requirements, schedule effect, cost effect, risk effect, quality effect, and decision.

Rejecting or deferring a change is not poor service; it is scope control.

Quality means the system meets requirements, is usable, reliable, secure enough for the context, and accepted by the right people.

Project success is not only being on time. It is fit-for-purpose delivery under controlled scope, cost, schedule, risk, and quality.

Real-life example — Student Tech Conference Registration Portal

Lesson 2 · §4.9

Task: evaluate the project plan, critical path, earned-value status, and risk register for a small registration portal.

Relation: shows how scope, schedule, cost, risk, role ownership, change control, and quality fit together.

Objective: launch a portal before the student technology conference.

Planned work: requirements, screen design, database build, integration, and user acceptance testing.

Current status: work is behind schedule and over budget in earned-value terms.

Management action: protect scope, focus on critical tasks, and act on highest risks.

Code Example 2 — Python project-control checker

Lesson 2 · Code

```
                                            # Calculate the planned project duration as the latest finish value.
planned_duration = max(earliest_finish.values())
# Add all planned values to obtain total PV.
planned_value = sum(task["pv"] for task in tasks)
# Add all earned values to obtain total EV.
earned_value = sum(task["ev"] for task in tasks)
# Add all actual costs to obtain total AC.
actual_cost = sum(task["ac"] for task in tasks)
# Calculate schedule variance and cost variance.
schedule_variance = earned_value - planned_value
cost_variance = earned_value - actual_cost
# Calculate performance indexes.
schedule_performance_index = earned_value / planned_value
cost_performance_index = earned_value / actual_cost
```

Task: calculate dependency-based duration and earned-value measures.

Relation: project status becomes objective indicators instead of opinion.

Full file: code/L2_project_control.py; every executable line is commented.

Output: earliest finish, duration, PV, EV, AC, SV, CV, SPI, and CPI.

Lesson 2 — checkpoint and candidate video

Lesson 2 · §4.10

Checkpoint questions:

1. Why does a project need scope control before a fair schedule can exist?
2. Which dependency path controls the Student Tech Conference Portal duration?
3. What do SPI = 0.76 and CPI = 0.88 mean?
4. Which risk has the highest exposure, and why?
5. Why can a project be on time and still fail?

Candidate short-video search: project management scope schedule cost risk short lecture on YouTube.

Verify any external video before distribution; link, do not copy or re-host.

LESSON 3

Managing Global Systems · What changes when the same system must operate across countries?

Read **Apply** **Assess**

What makes a system global

Lesson 3 · §5.1

A global system supports users, data, processes, laws, operations, or partners across more than one country or region.

Global complexity appears in privacy law, data location, language, formats, currencies, taxes, payment methods, network performance, time zones, support, and integration.

A global system is not automatically one identical system everywhere.

The management question: what must remain standard, and what may or must vary locally?

A good global design makes those decisions explicit before rollout.

Business strategy and system configuration

Lesson 3 · §5.2–5.3

Centralized: one central system and central decision authority.

Duplicated: common design copied into regions that operate local instances.

Decentralized: regions choose and manage their own systems.

Networked: shared global core with local extensions and coordinated governance.

The chosen configuration should match the organization’s strategy, legal obligations, operating model, and need for shared data.

Configuration	Decision authority	Best fit	Main risk
Centralized	Centre	Strong standardization/control	May ignore local needs
Duplicated	Centre designs; regions operate copies	Similar regions needing local resilience	Version drift
Decentralized	Regions	Different legal/market conditions	Inconsistent data/processes
Networked	Shared global core + local extensions	Global firms needing both consistency/local fit	Governance complexity

The global design decision — standard or local?

Lesson 3 · §5.4

Likely global core: identity, account structure, master customer/student data, main reporting definitions, security baseline, audit logging, and core integration standards.

Likely local variation: language, tax, currency, payment options, address formats, date/time formats, local content, support hours, and legally required notices.

Do not localize because a team prefers a different habit.

Do not standardize something that local law or market operation genuinely requires to vary.

Document every standard/local decision, owner, reason, and review date.

Data protection, privacy, and data location

Lesson 3 · §5.5

Privacy assessment must examine data flows, not only server location.

Questions: what personal data is collected, why, where it is stored, where it is accessed from, who receives it, how long it is kept, and how it is deleted.

Important official sources: Office of the Privacy Commissioner of Canada, Commission d’accès à l’information du Québec, and GDPR (General Data Protection Regulation) official text at EUR-Lex.

Data-location choices affect latency, legal review, contracts, access controls, incident response, backup, and support.

A global rollout should not proceed without a current data-flow map.

Language, culture, formats, and accessibility

Lesson 3 · §5.6

Localization is more than translation.

It includes terminology, date formats, time formats, decimal separators, address formats, name fields, reading direction, content ownership, examples, support scripts, and accessibility.

A literal translation can still fail if the workflow, terms, or examples do not match local practice.

Accessibility should be designed into the system and content, not added as an afterthought.

Assessment evidence: localized content inventory, user validation, and named owners for future updates.

Currency, payments, tax, and financial integration

Lesson 3 · §5.7

A global system that accepts payments or issues invoices must handle currency, payment methods, refund rules, tax rules, reconciliation, and financial reporting.

A payment method common in one market may be unusual or unavailable in another.

Currency conversion creates timing, rounding, refund, and reporting questions.

Tax handling often requires country-specific rules and evidence.

Financial integration must match accounting and audit requirements, not only checkout convenience.

Assessment evidence: finance sign-off and end-to-end test results.

Network performance, availability, and regional architecture

Lesson 3 · §5.8

Performance should be measured from the region where users actually work.

Global users may experience different response time even when the system is technically available.

Availability requirements should include support procedures, monitoring, backup, recovery, and escalation.

Regional architecture choices can include central hosting, regional replicas, content delivery, caching, or local services.

Assessment evidence: regional performance tests, monitoring baseline, and incident-response plan.

Time zones and support operations

Lesson 3 · §5.9

A global system is only operationally ready if users can receive help during the hours when they actually work.

A single central support window may not overlap enough with all regions.

The fictional support window is 13:00–21:00 UTC (Coordinated Universal Time).

Canada-East has 8 hours of overlap; Spain has 2 hours; Singapore has 0 hours.

Management action for Singapore: add Asia-Pacific support coverage or a follow-the-sun handoff before full rollout.

Operational readiness is not solved by software alone.

Region	UTC offset	Local workday	Central support UTC	Overlap	Finding
Canada-East	-4	09:00-17:00	13:00-21:00	8 h	Adequate
Spain	+2	09:00-17:00	13:00-21:00	2 h	Adequate minimum
Singapore	+8	09:00-17:00	13:00-21:00	0 h	Insufficient

Global governance, master data, and integration

Lesson 3 · §5.10

Global governance decides who may change shared definitions, master records, configuration, roles, reports, interfaces, and controls.

Master data must have common definitions for shared reporting and integration.

If regional teams redefine shared fields independently, global reports become inconsistent.

Integration should have documented APIs (Application Programming Interfaces), owners, monitoring, error handling, and support responsibility.

Governance must be strong enough to protect the global core and flexible enough to allow justified local variation.

Rollout strategy — pilot, waves, evidence, local readiness

Lesson 3 · §5.11

A rollout is the managed movement from design to real use.

Pilot: start in one limited location or group to test assumptions and evidence.

Wave rollout: deploy to groups or regions in planned stages.

Big bang: launch everywhere at once; faster but higher risk.

A global rollout should use readiness evidence, not political pressure or calendar optimism.

Readiness evidence includes privacy, language, payments, performance, support, integration, training, and acceptance results.

Criterion	Weight	Score 5 means
Privacy and regulation	0.25	Approved compliance path + data-flow map
Language and localization	0.15	Languages, formats, accessibility ready
Payments and finance	0.15	Currencies, invoicing, tax, reconciliation tested
Network performance	0.20	Target response time/availability achieved
Support coverage	0.15	Support during local working hours
Integration readiness	0.10	Interfaces documented/tested/monitored

Real-life example — Global Tutoring Platform Rollout

Lesson 3 · §5.12

Task: assess whether Canada-East, Spain, and Singapore are ready for pilot rollout.

Relation: applies privacy, localization, payments, performance, support, and integration criteria from the lesson.

Business goal: one tutoring platform across multiple regions with shared identity, reporting, and lesson records.

Key management problem: keep a reliable global core while allowing local differences that are legally or operationally required.

Weighted score threshold for “Ready for pilot” in this example: 4.00 / 5.00.

Do not approve rollout using a score alone; check mandatory gaps.

Schematic — global tutoring platform architecture

Lesson 3 · Figure

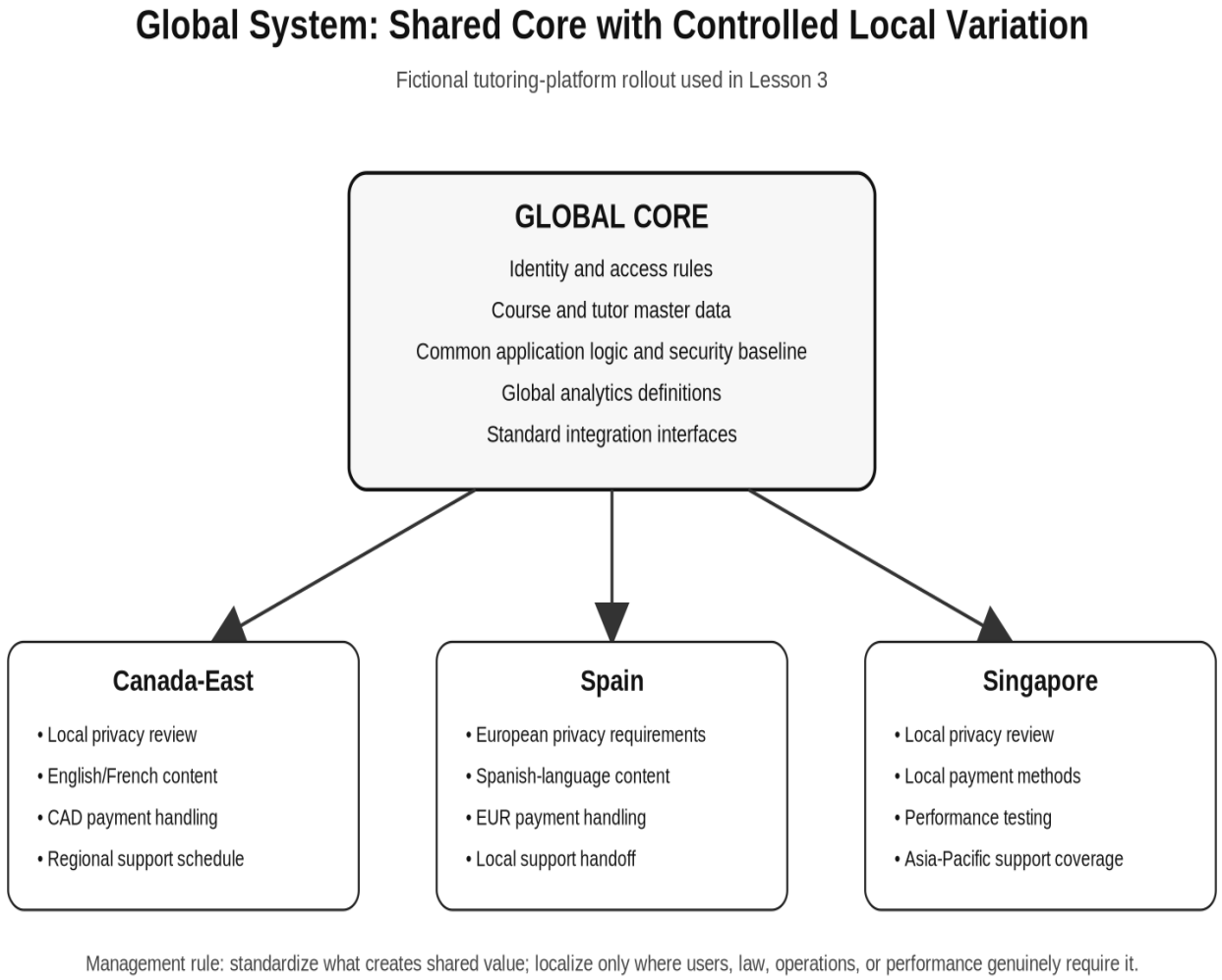
Global core: identity, lesson records, reporting definitions, audit logging, and integration standards.

Regional layer: language content, payments, tax/invoice rules, support hours, and local compliance evidence.

Integration layer: APIs, monitoring, error handling, and ownership across systems.

Governance protects shared definitions while allowing documented local variation.

The diagram is fictional and does not represent any vendor architecture.



Regional readiness scores

Lesson 3 · Spreadsheet

Canada-East score: 4.85 / 5.00 — Ready for pilot.

Spain score: 3.70 / 5.00 — Address gaps before pilot.

Singapore score: 3.50 / 5.00 — Address gaps before pilot.

Spain action: complete Spanish localization and European data-protection review.

Singapore action: add Asia-Pacific support coverage and repeat performance tests.

Management warning: a score does not override mandatory legal, financial, support, or integration blockers.

Region	Privacy	Lang.	Pay.	Net.	Support	Int.	Score	Recommendation
Canada-East	5	4	5	5	5	5	4.85	Ready for pilot
Spain	4	3	4	4	3	4	3.70	Address gaps before pilot
Singapore	4	4	4	3	2	4	3.50	Address gaps before pilot

Code Example 3 — Python global-rollout assessor

Lesson 3 · Code

```
                                                    # Define management weights for rollout criteria.
weights = {"privacy": 0.25, "language": 0.15, "payments": 0.15,
           "latency": 0.20, "support": 0.15, "integration": 0.10}
# Convert one regional record into a weighted score out of five.
def readiness_score(region):
    return sum(region[criterion] * weight for criterion, weight in weights.items())
# Calculate overlap between local workday and one UTC support window.
def support_overlap_hours(utc_offset):
    local_start_utc = 9 - utc_offset
    local_end_utc = 17 - utc_offset
    overlap_start = max(local_start_utc, 13)
    overlap_end = min(local_end_utc, 21)
    return max(0, overlap_end - overlap_start)
```

Task: calculate weighted readiness and support-window overlap for each region.

Relation: readiness and support coverage become explicit.

Full file: code/L3_global_rollout_assessor.py; every executable line is commented.

Output: score, support overlap, pilot recommendation, and operating-model gap.

Lesson 3 — checkpoint and candidate video

Lesson 3 · §5.13

Checkpoint questions:

1. Explain centralized versus decentralized global systems.
2. Give two examples of global core information and two examples needing regional variation.
3. Why is language localization more than translation?
4. Why must privacy assessment include data flows, not only server location?
5. What does the support-overlap example reveal about operational readiness?
6. Why can a high weighted readiness score still be insufficient for launch?

Candidate short-video search: global information systems management short lecture on YouTube.

Integrated recap — how the three lessons connect

Lecture §6

Building Information Systems defines the business problem, requirements, feasibility, design, testing, conversion, operation, and maintenance.

Managing Projects controls the delivery work through scope, schedule, cost, risk, quality, responsibility, and change control.

Managing Global Systems adds cross-country conditions: privacy, language, formats, currencies, payments, tax, network performance, support, time zones, governance, and integration.

The common assessment discipline: state the task, gather evidence, make assumptions visible, calculate where possible, and document the decision.

A technically correct system can still fail if it solves the wrong problem, arrives uncontrolled, or cannot operate in the locations where users need it.

Asynchronous preparation activity

Lecture §7

Scenario: a department wants a small student-service system that begins locally but may later be shared internationally.

Student task: prepare a one-page assessment using the three lesson lenses.

1. Building: define three requirements, one feasibility concern, and one release-one scope decision.
2. Project: list five tasks, at least three dependencies, one milestone, and two risks with owners.
3. Global: identify two items that should remain standard and two that may require localization.

Self-check: every claim should connect to a requirement, table, calculation, test, or documented assumption.

Official and safety links

Lecture §9

Python Software Foundation: https://www.python.org/
Official Python download and documentation.

Project Management Institute (PMI): https://www.pmi.org/
Professional body for project-management practice.

International Organization for Standardization (ISO): https://www.iso.org/
International standards organization.

ISO/IEC 27001 overview: https://www.iso.org/standard/27001
Information-security management standard overview.

Office of the Privacy Commissioner of Canada: https://www.priv.gc.ca/en/
Canadian federal privacy authority.

Commission d'accès à l'information du Québec: https://www.cai.gouv.qc.ca/

GDPR official text (EUR-Lex): https://eur-lex.europa.eu/eli/reg/2016/679/oj

VirusTotal URL scanner: https://www.virustotal.com/gui/home/url

Lesson 1 candidate search: https://www.youtube.com/results?search_query=building+information+systems+systems+development+life+cycle+short+lecture

Lesson 2 candidate search: https://www.youtube.com/results?search_query=project+management+scope+schedule+cost+risk+short+lecture

Lesson 3 candidate search: https://www.youtube.com/results?search_query=global+information+systems+management+short+lecture

Disclaimer, rights, and package index

Lecture §10–§11

Cyber.SoHo Educational Hub owns the original material created for this lecture package.

Third-party names, platforms, standards bodies, organizations, and websites remain the property of their respective owners.

No affiliation, sponsorship, endorsement, certification, or partnership is implied by references or links.

External videos and websites are linked only; they are not copied, embedded, cached, re-hosted, or redistributed.

Users should verify external URLs using a tool such as VirusTotal before opening them.

All examples, organizations, people, data, prices, scores, and events are fictional for teaching purposes.

Code and spreadsheets are educational examples only; do not deploy them in production.

This material is not legal, financial, technical, regulatory, security, or professional advice.